Laboratorium Testowania Oprogramowania

Listy zakupów

Plan testów i raport z testów

Autor:	Eryk Hadała
Grupa:	Grupa laboratoryjna 1
Prowadzący:	dr inż. Grzegorz Rogus
Data:	26 października 2026
Wersja aplikacji:	0.1.0 (gałąź `main`, stan roboczy)

Dokument składa się z dwóch części. **Plan testów** opisuje środowisko, narzędzia, przypadki testowe pięciu poziomów oraz dane testowe wraz z uzasadnieniem ich wyboru. **Raport z testów** zawiera wyniki wykonania, audyt bezpieczeństwa, analizę i zalecenia dla zespołu developerskiego.

Spis treści

I Plan testów **3**

1 Wprowadzenie **3**
1.1 Cel dokumentu . . . 3
1.2 Przedmiot testów . . . 3
1.3 Metodyka . . . 3

2 Środowisko testowe i narzędzia **4**
2.1 Sprzęt i system . . . 4
2.2 Oprogramowanie i narzędzia . . . 5
2.3 Konfiguracja środowisk . . . 5

3 Kryteria wejścia i wyjścia **5**

4 Testy jednostkowe — przypadki testowe **6**
4.1 Zakres i techniki . . . 6
4.2 Backend — reguły domeny . . . 6
4.3 Frontend — komponenty i logika klienta . . . 7

5 Testy integracyjne — przypadki testowe **8**

6 Dane do testów integracyjnych **9**
6.1 Zestaw danych . . . 9
6.2 Dlaczego taki zestaw . . . 10

7 Testy systemowe — przypadki testowe **10**

8 Dane do testów systemowych **11**

9 Testy wydajnościowe — przypadki testowe **12**
9.1 Model obciążenia . . . 12
9.2 Dane . . . 12

10 Audyt bezpieczeństwa — plan **12**
10.1 Zakres i metody . . . 12
10.2 Przegląd kodu — lista kontrolna . . . 13

II Raport z testów **14**

11 Podsumowanie wykonania **14**

12 Wyniki testów jednostkowych i integracyjnych **14**

12.1 Wykonanie 14

12.2 Pokrycie kodu 14

12.3 Obszary o słabszym pokryciu 15

13 Wyniki testów systemowych **15**

14 Wyniki testów wydajnościowych **16**

14.1 Analiza 16

15 Wyniki audytu bezpieczeństwa **17**

15.1 Testy automatyczne i zależności 17

15.2 OWASP ZAP baseline 17

15.3 Ustalenia przeglądu kodu 17

15.4 Mechanizmy ocenione pozytywnie 18

16 Defekty wykryte w trakcie testów **18**

17 Wnioski i zalecenia dla zespołu developerskiego **18**

17.1 Wnioski 18

17.2 Zalecenia 19

A Odtworzenie wyników **20**

CZĘŚĆ I

Plan testów

1 Wprowadzenie

1.1 Cel dokumentu

Plan określa, co, czym i na jakich danych jest testowane w aplikacji „Listy zakupów". Każdy przypadek ma identyfikator, z którego korzysta część raportowa: **TJ** — testy jednostkowe, **TI** — integracyjne, **TS** — systemowe, **TW** — wydajnościowe, **TB** — bezpieczeństwa.

1.2 Przedmiot testów

Aplikacja webowa dostępna wyłącznie po zalogowaniu. Użytkownik zakłada konto, tworzy listy zakupów z sugerowaną datą i dodaje pozycje. Listę można udostępnić innej osobie, która może jedynie zmieniać status pozycji. Frontend: React, Vite, shadcn/ui. Backend: Node.js, Hono, SQLite z ręcznie pisanymi zapytaniami parametryzowanymi. Zdjęcia pozycji są zapisywane w bazie jako BLOB.

ID	Wymaganie
R1	Rejestracja, logowanie i wylogowanie. Bez sesji aplikacja i API są niedostępne.
R2	Pełny CRUD list zakupów. Każda lista ma opcjonalną sugerowaną datę zakupów.
R3	Pozycja ma nazwę wpisaną ręcznie albo wybraną z popularnych produktów (chleb, mleko, woda, ser…), ilość dodatnią i jednostkę z listy `szt`, `g`, `kg`, `ml`, `l`, `opak`.
R4	Pozycja może mieć zdjęcie referencyjne JPEG, PNG lub WebP do 2 MB.
R5	Pozycja kupiona jest na liście przekreślona.
R6	Właściciel udostępnia listę po adresie e-mail. Osoba ze współdzielenia zmienia tylko status, bez dodawania, usuwania i zmiany ilości.
R7	Listy, pozycje, zdjęcia i statusy są trwale zapisane w bazie.
R8	Wymagania niefunkcjonalne: p95 czasu odpowiedzi poniżej 300 ms, mniej niż 1% błędów, brak podatności o wadze wysokiej.

Tabela 1: Wymagania objęte testami

1.3 Metodyka

Aplikacja powstaje metodyką **TDD** w cyklu czerwony — zielony — refaktor. Najpierw powstaje test opisujący jedną regułę i pada, potem najmniejszy kod, który go spełnia, na końcu porządkowanie przy zielonym teście. Cykle zaczynają się od reguł domeny bez bazy i HTTP, przechodzą przez API na bazie w pamięci, a kończą na komponentach interfejsu. Test systemowy w przeglądarce jest testem akceptacyjnym całej ścieżki. Testy wydajności, testy bezpieczeństwa i skan DAST badają złożoną aplikację i nie są krokiem wewnętrznego cyklu TDD.

Poziom	Narzędzie	Zakres	Polecenie
Jednostkowy	Vitest, Testing Library	reguły domeny, komponenty React	`pnpm test:unit`
Integracyjny	Vitest, `app.request()`	trasa HTTP → serwis → repozytorium → SQLite	`pnpm test:integration`
Systemowy	Playwright, Chromium	cała aplikacja w przeglądarce, dwóch użytkowników	`pnpm test:e2e`
Wydajnościowy	k6	logowanie, odczyt list, zmiana statusu pod obciążeniem	`pnpm test:perf`
Bezpieczeństwa	Vitest, pnpm audit, OWASP ZAP	ataki na API, zależności, skan pasywny	`pnpm test:security` `pnpm test:zap`

Tabela 2: Poziomy testów i narzędzia

2 Środowisko testowe i narzędzia

2.1 Sprzęt i system

Element	Wartość
System operacyjny	CachyOS Linux, jądro 6.18.50-2-cachyos-lts, x86_64
Procesor	12 wątków
Pamięć RAM	15 GiB
Sieć	pętla zwrotna `127.0.0.1`; klient i serwer działają na tej samej maszynie
Konteneryzacja	Docker 29.9.0 (tylko dla OWASP ZAP)

Tabela 3: Platforma sprzętowa

2.2 Oprogramowanie i narzędzia

Narzędzie	Wersja	Rola
Node.js	26.10.0	środowisko uruchomieniowe API, testów i serwera deweloperskiego
pnpm	12.10.1	menedżer pakietów i workspace, uruchamianie skryptów testowych
Hono	4.13.13	framework HTTP; `app.request()` pozwala testować API bez gniazda sieciowego
better-sqlite3	12.11.1	sterownik SQLite; baza `:memory:` w testach, plik WAL w pozostałych środowiskach
Vitest	4.1.11	testy jednostkowe, integracyjne i bezpieczeństwa
@vitest/coverage-v8	4.1.11	pomiar pokrycia kodu i egzekwowanie progów
Testing Library	16.3.3	render komponentów React i symulacja działań użytkownika
jsdom	27.4.0	przeglądarkowe DOM w Node.js dla testów komponentów
Playwright	1.64.0	testy systemowe; Chromium build 1248
k6	2.3.0	testy wydajnościowe smoke, load, stress
OWASP ZAP	2.17.0	pasywny skan bezpieczeństwa (obraz `zaproxy/zap-stable`)
pnpm audit	12.10.1	audyt znanych podatności w zależnościach
TypeScript	5.9	kontrola typów jako dodatkowa bramka jakości

Tabela 4: Oprogramowanie użyte do testów

2.3 Konfiguracja środowisk

Każdy poziom testów ma osobną bazę danych, dzięki czemu przebiegi nie wpływają na siebie i na dane deweloperskie.

Środowisko	Adres	Baza	Ustawienia
Jednostkowe i integracyjne	brak gniazda	`:memory:`	nowa baza na każdy test; limit logowań 1000, ciasteczka bez `Secure`
Systemowe	`:3000` API, `:5173` front	`e2e.sqlite`	plik usuwany przed przebiegiem; serwery stawia Playwright
Wydajnościowe	`:3099`	`perf.sqlite`	plik tworzony od zera; `LOGIN_RATE_LIMIT_MAX=100000`
Skan ZAP	`:3000`	`app.sqlite`	kontener w sieci hosta, bo API nasłuchuje tylko na `127.0.0.1`

Tabela 5: Konfiguracja środowisk testowych

3 Kryteria wejścia i wyjścia

Kryteria wejścia: zależności zainstalowane (`pnpm install`), kod przechodzi kontrolę typów, dla testów systemowych zainstalowana przeglądarka Chromium, dla testów wydajności dostępny k6, dla skanu ZAP działający Docker.

Kryteria wyjścia:

1. wszystkie przypadki TJ, TI, TS i TB zakończone wynikiem pozytywnym;

2. pokrycie kodu nie niższe niż progi zapisane w konfiguracji Vitest: API 90% instrukcji, 85% gałęzi, 90% funkcji, 95% linii; front 90% instrukcji, 80% gałęzi, 90% funkcji, 90% linii;
3. w scenariuszach TW: p95 czasu odpowiedzi poniżej 300 ms i odsetek błędów poniżej 1%;
4. brak podatności o wadze *high* lub *critical* w `pnpm audit`;
5. brak alertów typu FAIL w skanie OWASP ZAP baseline.

4 Testy jednostkowe — przypadki testowe

4.1 Zakres i techniki

Testy jednostkowe sprawdzają czystą logikę bez bazy i bez sieci oraz komponenty interfejsu w izolacji od backendu (żądania `fetch` są zastępowane atrapami). Do projektowania danych użyto **klas równoważności** (poprawna i niepoprawna jednostka), **wartości brzegowych** (ilość 0 i 1 000 001, rozmiar zdjęcia 2 MB + 1 bajt, hasło 7 znaków) oraz **tablicy decyzyjnej** dla uprawnień (tabela 6).

Działanie	Właściciel	Osoba ze współdzielenia	Obca osoba
Podgląd listy	tak	tak	nie (404)
Zmiana statusu pozycji	tak	tak	nie (404)
Dodanie, edycja, usunięcie pozycji	tak	nie (403)	nie (404)
Edycja i usunięcie listy, udostępnianie	tak	nie (403)	nie (404)

Tabela 6: Tablica decyzyjna uprawnień (R6)

4.2 Backend — reguły domeny

ID	Przypadek	Dane wejściowe	Oczekiwany wynik
apps/api/src/domain/units.test.ts			
TJ-01	Jednostka z listy (R3)	każda z `szt`, `g`, `kg`, `ml`, `l`, `opak`; `tona`; pusty napis	jednostki z listy przyjęte, `tona` i pusty napis odrzucone
TJ-02	Wartości brzegowe ilości (R3)	1; 1,5; 0; -2; NaN; $+\infty$; 1 000 001	1 i 1,5 przyjęte, pozostałe odrzucone
apps/api/src/domain/schemas.test.ts			
TJ-03	Walidacja rejestracji (R1)	hasło `haslo1234` (9 znaków); hasło `krotkie` (7 znaków)	pierwsze przyjęte, drugie odrzucone
TJ-04	Data listy (R2)	`2026-02-31`; `2026-10-12`; pusta zmiana listy	nieistniejąca data i pusta zmiana odrzucone, poprawna data przyjęta
TJ-05	Schemat pozycji (R3)	ilość `"2"` jako tekst z formularza; ilość 0; jednostka `tona`	tekst zamieniony na liczbę i przyjęty, pozostałe odrzucone
apps/api/src/domain/items.test.ts			
TJ-06	Kolejność pozycji (R3)	trzy pozycje: pozycja 2; pozycja 1 z lutego; pozycja 1 ze stycznia	kolejność `a`, `c`, `b`: po pozycji, przy remisie po dacie
TJ-07	Podsumowanie listy (R5)	statusy: kupione, niekupione, kupione	razem 3, kupione 2, pozostało 1
apps/api/src/domain/rateLimit.test.ts			

ID	Przypadek	Dane wejściowe	Oczekiwany wynik
TJ-08	Okno limitu prób (R8)	limit 2, okno 500 ms; próby w chwilach 1000, 1100, 1200, 1600 ms	dwie pierwsze dozwolone, trzecia zablokowana, czwarta po resecie okna dozwolona
apps/api/src/domain/image.test.ts			
TJ-09	Poprawny PNG (R4)	plik PNG 1×1 piksel, 70 B	przyjęty
TJ-10	Niepoprawne zdjęcie (R4)	typ `text/plain`; bajty `01 02 03 04` jako PNG; pusty plik; PNG o rozmiarze 2 MB + 1 B	wszystkie odrzucone z komunikatem
TJ-11	Podpisy JPEG i WebP (R4)	`FF D8 FF`; `00 00 00` jako JPEG; nagłówek `RIFF…WEBP`; ucięty WebP	poprawne podpisy przyjęte, błędne odrzucone
apps/api/src/domain/permissions.test.ts			
TJ-12	Prawa właściciela (R6)	właściciel listy	może edytować pozycje i zmieniać status
TJ-13	Prawa osoby ze współdzielenia (R6)	użytkownik z listy udostępnień	zmienia tylko status, edycja zabroniona
TJ-14	Obca osoba (R6)	użytkownik spoza listy	brak podglądu, edycji i zmiany statusu

4.3 Frontend — komponenty i logika klienta

ID	Przypadek	Dane wejściowe	Oczekiwany wynik
apps/web/src/components/ItemRow.test.tsx			
TJ-15	Przekreślenie kupionej pozycji (R5)	pozycja `isPurchased: true`	etykieta ma klasę `line-through`
TJ-16	Widok osoby ze współdzielenia (R6)	`canEdit: false`	brak przycisków edycji i usuwania
TJ-17	Zgłoszenie zmiany statusu (R5)	kliknięcie pola wyboru	`onToggle(true)`
apps/web/src/components/AddItemForm.test.tsx			
TJ-18	Dodanie pozycji z domyślną jednostką (R3)	nazwa `mleko`, ilość 2	wysłane `{mleko, 2, szt, bez zdjęcia}`
TJ-19	Wybór produktu z podpowiedzi i zdjęcie (R3, R4)	wpis `mle`, wybór `mleko`, plik PNG	wysłane `mleko`, ilość 1 i plik
TJ-20	Nazwa spoza katalogu (R3)	wpis `tofu`, opcja „Dodaj „tofu””	wysłana nazwa `tofu`
TJ-21	Pusta nazwa (R3)	wysłanie pustego formularza	brak wysyłki, widoczny komunikat błędu
apps/web/src/components/ProtectedRoute.test.tsx			
TJ-22	Strona chroniona bez sesji (R1)	brak użytkownika	przekierowanie na logowanie
TJ-23	Strona chroniona z sesją (R1)	zalogowany Jan	widoczna treść list
TJ-24	Sprawdzanie sesji (R1)	`loading: true`	wskaźnik ładowania
TJ-25	Logowanie dla gościa (R1)	brak użytkownika na `/login`	widoczny ekran logowania
TJ-26	Logowanie przy aktywnej sesji (R1)	zalogowany Jan na `/login`	przekierowanie na listy
apps/web/src/components/DateField.test.tsx			
TJ-27	Formatowanie daty (R2)	`null`; `wtorek`; `2026-10-16`	„Bez daty”; tekst bez zmian; data po polsku
TJ-28	Wybór daty polem i kalendarzem (R2)	wpis `2026-10-20`; kliknięcie dnia w kalendarzu	`onChange` wywołane w obu przypadkach
apps/web/src/components/ShareDialog.test.tsx			

ID	Przypadek	Dane wejściowe	Oczekiwany wynik
TJ-29	Udostępnienie i odebranie dostępu (R6)	e-mail `ania@example.com`, przycisk „Odbierz”	`onShare` z adresem, `onUnshare('2')`
TJ-30	Nieudane udostępnienie (R6)	`onShare` rzuca błąd	adres zostaje w polu
apps/web/src/components/AppHeader.test.tsx			
TJ-31	Wylogowanie (R1)	kliknięcie „Wyloguj”	wywołane `logout`, przejście na logowanie
apps/web/src/auth/AuthProvider.test.tsx			
TJ-32	Cykl sesji w kliencie (R1)	odpowiedzi `me`, `login`, `register`, `logout`	nazwa użytkownika zmienia się po każdym kroku, na końcu gość
TJ-33	Brak sesji (R1)	`/api/auth/me` zwraca 401	stan gościa
TJ-34	Użycie poza providerem	render bez `AuthProvider`	błąd „useAuth poza AuthProvider”
apps/web/src/lib/api.test.ts, apps/web/src/lib/schemas.test.ts			
TJ-35	Nagłówek JSON	ciało tekstowe	`content-type: application/json`, zwrócone ciało
TJ-36	Formularz i pusta odpowiedź	`FormData`, odpowiedź 204	brak nadpisanego typu, wynik `undefined`
TJ-37	Obsługa błędów API	400 z `error`; 500 bez JSON	komunikat z serwera; komunikat ogólny
TJ-38	Schemat pozycji w kliencie (R3)	jednostka `tona`	odrzucona
apps/web/src/pages/auth-pages.test.tsx			
TJ-39	Udane logowanie (R1)	`jan@example.com` / `haslo1234`	wywołane `login`, przejście na listy
TJ-40	Błędne hasło (R1)	`login` rzuca `ApiError(401)`	widoczny komunikat z serwera
TJ-41	Pusty formularz logowania (R1)	brak danych	brak wywołania `login`
TJ-42	Udana rejestracja (R1)	Jan, e-mail, hasło 9 znaków	wywołane `register`, przejście na listy
TJ-43	Błąd sieci przy rejestracji (R1)	zwykły `Error`	komunikat „Nie udało się utworzyć konta”
apps/web/src/pages/lists-flow.test.tsx, apps/web/src/App.test.tsx			
TJ-44	Listy własne i udostępnione (R2, R6)	listy `Piątek` i `Od Ani`, nowa `Sobota`	obie sekcje widoczne; po utworzeniu widok nowej, pustej listy
TJ-45	Pusty stan (R2)	brak list	komunikaty pustych sekcji
TJ-46	Pełna obsługa listy przez właściciela (R2–R6)	odhaczenie, dodanie `chleb`, udostępnienie, zmiana nazwy, edycja i usunięcie pozycji, usunięcie listy	każde działanie wysyła żądanie, na końcu powrót do list
TJ-47	Błąd wczytania listy	odpowiedź 500	komunikat „Nie udało się otworzyć listy.”
TJ-48	Widok osoby ze współdzielenia (R6)	rola `shared`, zmiana statusu kończy się 500	notka o prawach, brak edycji, interfejs wraca do stanu sprzed zmiany
TJ-49	Nieznana ścieżka bez sesji (R1)	adres `/nie-ma`, `me` zwraca 401	ekran logowania

5 Testy integracyjne — przypadki testowe

Testy integracyjne wysyłają żądania HTTP do aplikacji Hono przez `app.request()`. Żądanie przechodzi przez middleware (nagłówki, limit rozmiaru, sesja), trasę, serwis i repozytorium do prawdziwej bazy SQLite w pamięci z pełnym schematem. Repozytoria nie mają osobnych testów: ich

zachowanie wymuszają poniższe przypadki.

ID	Przypadek	Dane wejściowe	Oczekiwany wynik
apps/api/src/integration/auth.test.ts			
TI-01	Rejestracja, sesja i wylogowanie (R1)	Jan, `jan@example.com`, `haslo1234`	201 i ciasteczko `session`; `/me` 200; wylogowanie 204; `/me` 401
TI-02	Brak wycieku istnienia konta (R1)	nieznany e-mail; znany e-mail ze złym hasłem	oba 401 z identyczną treścią
TI-03	Unikalny e-mail bez względu na wielkość liter (R1)	druga rejestracja `Jan@example.com`	409
TI-04	Wymóg sesji (R1)	`GET /api/lists` bez ciasteczka	401
apps/api/src/integration/lists.test.ts			
TI-05	Pełny CRUD listy i pozycji ze zdjęciem (R2–R5, R7)	lista `Zakupy na piątek` 2026-10-16; pozycja `mleko` 2 l z PNG w `multipart`	201; zmiana nazwy; zdjęcie zwrócone bajt w bajt z typem `image/png`; edycja; status kupione; usunięcie 204; lista 404
TI-06	Kaskadowe usuwanie (R7)	lista z pozycją `chleb`	po usunięciu listy w tabeli `items` 0 wierszy
TI-07	Katalog i jednostki (R3)	`/popular-products` bez sesji i z sesją; `/units`	401; lista zawiera `chleb`; jednostki dokładnie `szt, g, kg, ml, l, opak`
apps/api/src/integration/sharing.test.ts			
TI-08	Prawa osoby ze współdzielenia (R6)	Jan udostępnia listę z pozycją `woda` 6 szt Ani	Ania: dodanie, edycja, usunięcie 403; zmiana statusu 200; Jan widzi status kupione i licznik 1
TI-09	Obca lista i udostępnienie sobie (R6)	Obcy otwiera listę Jana; Jan udostępnia sam sobie	404; 400
apps/api/src/integration/edges.test.ts			
TI-10	Sprawdzenie stanu i nieznana ścieżka	`/api/health`; `/api/brak`	`{ok: true}`; 404
TI-11	Logowanie i ciasteczko `Secure` (R1)	logowanie istniejącego konta przy `secureCookies`	200 i flaga `Secure`; wylogowanie bez ciasteczka 204
TI-12	Złe ciało żądania i limit przez proxy (R1, R8)	ciało `null`; JSON `{`; limit 1 i `X-Forwarded-For: 10.0.0.8`	400; 400; druga próba z tego adresu 429
TI-13	Za duże żądanie i awaria bazy	ciało 3 MB + 1 B; zamknięta baza	413; 500 z ogólnym komunikatem
TI-14	Zarządzanie udostępnieniami (R2, R6)	usunięcie daty; udostępnienie nieznanemu; ponowne udostępnienie; odebranie przez Anię i przez Jana	data `null`; 404; 409; 403; 204, ponownie 404
TI-15	Wymiana zdjęcia (R4)	brak zdjęcia; pusty formularz; PNG; usunięcie przez Anię i Jana; zdjęcie 2 MB + 1 B	404; 400; `hasImage: true`; 403; 204; 400
TI-16	Plikowa baza i seed (R7)	nowy katalog tymczasowy, dwukrotny seed	tryb WAL; produkty zasiane, drugi seed nie dubluje danych

6 Dane do testów integracyjnych

6.1 Zestaw danych

Kategoria	Dane	Uzasadnienie
Baza	SQLite `:memory:` ze schematem produkcyjnym i 20 popularnymi produktami	każdy test zaczyna od czystego stanu; ten sam schemat i te same ograniczenia `CHECK` co w produkcji
Użytkownicy	Jan `jan@example.com` (właściciel), Ania `ania@example.com` (współdzielenie), Obcy `obcy@example.com`	po jednym przedstawicielu każdej roli z tablicy decyzyjnej uprawnień
Hasło	`haslo1234` (9 znaków)	minimalnie powyżej granicy 8 znaków; poprawne dla walidatora
Wielkość liter	`Jan@example.com`	sprawdza normalizację e-maila przy unikalności konta
Listy	`Zakupy na piątek`, data 2026-10-16; `Na dziś` bez daty	obie klasy: lista z datą i bez daty
Pozycje	`mleko` 2 l, `woda` 6 szt, `ser` 1 opak, `chleb` 1 szt	produkty z katalogu i różne jednostki, w tym niedomyślne
Zdjęcie	PNG 1×1 piksel, 70 B	najmniejszy poprawny plik; szybkie testy i porównanie bajt w bajt
Wartości brzegowe	zdjęcie 2 MB + 1 B; ciało 3 MB + 1 B	pierwszy bajt ponad limit zdjęcia i limit ciała żądania
Złe dane	ciało `null`, JSON `{`, nieznany e-mail, nieznane ID listy	ścieżki błędów, które klient może wywołać
Proxy	`X-Forwarded-For: 10.0.0.8, 10.0.0.1`, limit logowań 1	pokazuje, od czego zależy klucz limitu

Tabela 7: Dane do testów integracyjnych

6.2 Dlaczego taki zestaw

- **Izolacja i powtarzalność.** Baza w pamięci powstaje dla każdego testu, więc kolejność wykonania nie ma znaczenia, a cały zestaw trwa około 3 s.
- **Prawdziwy silnik zamiast atrapy.** Zapytania SQL, klucze obce i `ON DELETE CASCADE` wykonuje SQLite, więc test TI-06 sprawdza rzeczywiste zachowanie bazy, a nie założenie o nim.
- **Pokrycie ról.** Trzy konta odpowiadają trzem kolumnom tablicy decyzyjnej; mniej kont nie pozwoliłoby odróżnić odpowiedzi 403 od 404.
- **Wartości brzegowe zamiast losowych.** Pliki i ciała żądań są o jeden bajt większe od limitu, bo tam najczęściej powstają błędy typu „off by one”.
- **Czytelność.** Polskie nazwy produktów z katalogu odpowiadają realnemu użyciu aplikacji i ułatwiają analizę nieudanego testu.

7 Testy systemowe — przypadki testowe

Testy systemowe uruchamiają rzeczywiste API i serwer frontendu, a następnie sterują przeglądarką Chromium jak użytkownik. Scenariusz TS-02 używa dwóch niezależnych kontekstów przeglądarki, czyli dwóch osobnych sesji.

ID	Przypadek	Dane wejściowe	Oczekiwany wynik
TS-01	Logowanie istniejącym kontem (R1)	Ola, `login.<znacznik czasu>@example.com`, `haslo1234`	po rejestracji i wylogowaniu ponowne logowanie otwiera stronę list
TS-02	Właściciel zarządza listą, osoba ze współdzielenia zmienia tylko status (R1–R7)	Jan i Ania z unikalnymi adresami; lista `Zakupy na piątek`, 2026-10-16; `mleko` 2 l z `pixel.png`	kroki i oczekiwane wyniki w tabeli 8
TS-03	Weryfikacja ręczna w przeglądarce (R1–R5, R7)	konto `laboratorium@example.com`; lista `Zakupy na piątek` wybrana kalendarzem; `mleko` 2 l	rejestracja, lista z datą, przekreślenie, stan 1/1 po odświeżeniu; `/lists` po wylogowaniu przekierowuje na logowanie

Krok	Kto	Działanie	Oczekiwany wynik
1	Ania, Jan	rejestracja w osobnych kontekstach	obie osoby widzą stronę list
2	Jan	nowa lista, otwarcie kalendarza, data 2026-10-16	kalendarz widoczny; tytuł `Zakupy na piątek`
3	Jan	pozycja `mleko`, ilość 2, jednostka `l`, zdjęcie	pozycja z miniaturą zdjęcia
4	Jan	odhaczenie pozycji	etykieta przekreślona (`line-through`)
5	Jan	zmiana nazwy na `Zakupy na sobotę`	nowy tytuł
6	Jan	udostępnienie Ani po e-mailu	adres Ani na liście udostępnień
7	Ania	otwarcie udostępnionej listy	notka o prawach; brak formularza dodawania i przycisku usuwania
8	Ania	zmiana statusu pozycji	przekreślenie znika
9	Jan	odświeżenie strony	widzi zmianę Ani: pozycja nieprzekreślona
10	Jan	usunięcie pozycji z potwierdzeniem	lista bez pozycji
11	Jan	usunięcie listy z potwierdzeniem	powrót do list; listy `Zakupy na sobotę` brak

Tabela 8: Kroki scenariusza TS-02

8 Dane do testów systemowych

Kategoria	Dane	Uzasadnienie
Baza	`e2e.sqlite` usuwany przez `global-setup.ts` przed przebiegiem	start z czystą bazą, bez danych z poprzednich uruchomień
Konta	`jan.<czas>@`, `ania.<czas>@`, `login.<czas>@example.com`	unikalne adresy pozwalają uruchamiać testy wielokrotnie bez konfliktu 409
Hasło	`haslo1234`	poprawne według reguły 8 znaków, takie samo jak w testach integracyjnych
Lista	`Zakupy na piątek` → `Zakupy na sobotę`, data 2026-10-16	polskie znaki diakrytyczne sprawdzają kodowanie w całym łańcuchu; zmiana nazwy sprawdza edycję
Pozycja	`mleko`, 2, `l`	produkt z katalogu podpowiedzi; jednostka inna niż domyślna `szt` wymusza użycie listy rozwijanej
Zdjęcie	`fixtures/pixel.png`, 70 B	ścieżka przesłania pliku przez formularz bez spowalniania testu

Kategoria	Dane	Uzasadnienie
Dwie sesje	dwa konteksty Playwright	jedyny sposób, by w jednym teście zobaczyć prawa obu ról i propagację zmian

Tabela 9: Dane do testów systemowych

Dane odwzorowują typowe użycie aplikacji: jedna osoba planuje zakupy, druga odhacza produkty w sklepie. Wybrano najkrótszą ścieżkę, która przechodzi przez każde wymaganie funkcjonalne R1–R7. Przypadki brzegowe i błędne zostały sprawdzone niżej w piramidzie, gdzie są szybsze i stabilniejsze.

9 Testy wydajnościowe — przypadki testowe

9.1 Model obciążenia

Każdy scenariusz k6 loguje się raz w fazie `setup`, a następnie w pętli wykonuje `GET /api/lists` oraz `PATCH /api/items/:id/status`. Osobna grupa wirtualnych użytkowników (VU) loguje się cyklicznie, bo logowanie liczy skrót bcrypt i jest najdroższą operacją API. Skrypt `tests/perf/run.sh` uruchamia API na porcie 3099 na nowej bazie, seeduje dane, wykonuje trzy scenariusze i zatrzymuje serwer.

ID	Przypadek	Dane wejściowe	Oczekiwany wynik
TW-01	Smoke — czy system działa pod minimalnym ruchem (R8)	1 VU przez 10 s; przerwa 0,2 s	p95 $<$ 300 ms; błędy $<$ 1%; wszystkie sprawdzenia 200
TW-02	Load — typowe obciążenie (R8)	8 VU przegląda i odhacza, 2 VU loguje się; 20 s	jak wyżej
TW-03	Stress — rosnące obciążenie (R8)	przegląd: 0 $\rightarrow$ 15 $\rightarrow$ 30 $\rightarrow$ 0 VU; logowanie: 0 $\rightarrow$ 4 $\rightarrow$ 0 VU; 25 s	jak wyżej

9.2 Dane

Seed `tests/perf/seed.mjs` tworzy konto `perf@example.com` oraz 5 list po 10 pozycji, razem 50 pozycji. Odpowiedź `GET /api/lists` zawiera więc liczniki wyliczane zapytaniem agregującym na realnej liczbie wierszy. Limit logowań w tym środowisku podniesiono do 100 000 na minutę. Przy domyślnych 10 próbach na minutę scenariusze mierzyłyby odpowiedzi 429 zamiast wydajności.

10 Audyt bezpieczeństwa — plan

10.1 Zakres i metody

Audyt obejmuje API i zależności projektu. Wykorzystuje cztery metody:

1. **testy automatyczne ataków** (Vitest, przypadki TB-01–TB-07);
2. **audyt zależności** `pnpm audit --audit-level=high`;
3. **skan pasywny DAST** OWASP ZAP baseline na działającym API;
4. **przegląd kodu** pod kątem OWASP Top 10 (2021).

ID	Przypadek	Dane wejściowe	Oczekiwany wynik
TB-01	Nagłówki ochronne i flagi ciasteczka	`/api/health`; rejestracja przy `secureCookies`	`X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`; ciasteczko `HttpOnly`, `SameSite=Lax`, `Secure`
TB-02	Dostęp bez sesji	`/api/lists`, `/api/auth/me`, `/api/popular-products`	401 dla każdego
TB-03	Zgadywanie hasła	4 próby ze złym hasłem przy limicie 3	3 razy 401, czwarta 429
TB-04	Wstrzyknięcie SQL	nazwa listy `'; DROP TABLE lists; --`; e-mail `jan@example.com' OR '1'='1`	nazwa zapisana jako tekst; logowanie 400; tabela i konto nienaruszone
TB-05	XSS zapisany w danych	nazwa pozycji `<img src=x onerror=alert(1)>`	odpowiedź JSON, wartość bez zmian, brak dokumentu HTML
TB-06	IDOR — cudze zasoby	Obcy pyta o listę, zdjęcie i status pozycji Jana	404 dla każdego; właściciel dalej ma dostęp
TB-07	Złośliwe przesłanie pliku	`text/html`; tekst udający PNG; PNG 2 MB + 1 B	400 dla każdego; komunikat o limicie 2 MB

10.2 Przegląd kodu — lista kontrolna

Kategoria OWASP	Co jest sprawdzane	Metoda
A01 Broken Access Control	kontrola roli przy każdej operacji, 404 dla obcych	TB-06, TI-08, TI-09, przegląd
A02 Cryptographic Failures	bcrypt, losowość tokenu sesji, przechowywanie tokenu	przegląd
A03 Injection	zapytania parametryzowane, odpowiedzi JSON	TB-04, TB-05, przegląd
A04 Insecure Design	limit prób logowania, jego klucz i zasięg	TB-03, TI-12, przegląd
A05 Security Misconfiguration	nagłówki, cache, ujawnianie błędów	TB-01, TI-13, ZAP
A06 Vulnerable Components	znane podatności zależności	`pnpm audit`
A07 Identification and Authentication Failures	sesje, wylogowanie, komunikaty logowania	TB-02, TI-01, TI-02
A08 Software and Data Integrity	weryfikacja treści plików, polityka łańcucha dostaw	TB-07, przegląd
A09 Logging and Monitoring	rejestrowanie zdarzeń bezpieczeństwa	przegląd
A10 SSRF	wychodzące żądania serwera	przegląd (brak takich żądań)

Tabela 10: Lista kontrolna przeglądu kodu

CZĘŚĆ II

Raport z testów

11 Podsumowanie wykonania

Wszystkie testy wykonano 12 października 2026 w środowisku z rozdziału 2.

Poziom	Przypadki	Zal.	Niezal.	Czas	Wynik
Jednostkowe — backend	14	14	0	5,8 s[a]	zaliczony
Integracyjne	16	16	0		zaliczony
Jednostkowe — frontend	35	35	0	6,1 s	zaliczony
Systemowe (automatyczne)	2	2	0	4,7 s	zaliczony
Systemowe (ręczne)	1	1	0	—	zaliczony po poprawce D5
Wydajnościowe	3	3	0	57 s	zaliczony
Bezpieczeństwa (Vitest)	7	7	0	1,0 s	zaliczony
`pnpm audit`	1	1	0	< 1 s	zaliczony: 0 podatności
OWASP ZAP baseline	67 reguł	66	0 FAIL	ok. 2 min	zaliczony: 1 ostrzeżenie
Razem testy	**78**	**78**	**0**		zaliczony

[a] Testy jednostkowe backendu i integracyjne wykonują się razem w przebiegu z pomiarem pokrycia.

Tabela 11: Zbiorcze wyniki

Werdykt

Aplikacja spełnia wszystkie kryteria wyjścia z rozdziału 3. Wymagania funkcjonalne R1–R7 są potwierdzone na trzech poziomach testów, a wymaganie R8 przy obciążeniu do 34 wirtualnych użytkowników. Przegląd kodu wykazał dwa ustalenia o wadze średniej (B1, B2), które należy usunąć przed wdrożeniem produkcyjnym.

12 Wyniki testów jednostkowych i integracyjnych

12.1 Wykonanie

Wszystkie 65 przypadków TJ-01–TJ-49 i TI-01–TI-16 zakończyło się wynikiem pozytywnym (30 testów w 10 plikach dla API, 35 testów w 12 plikach dla frontu).

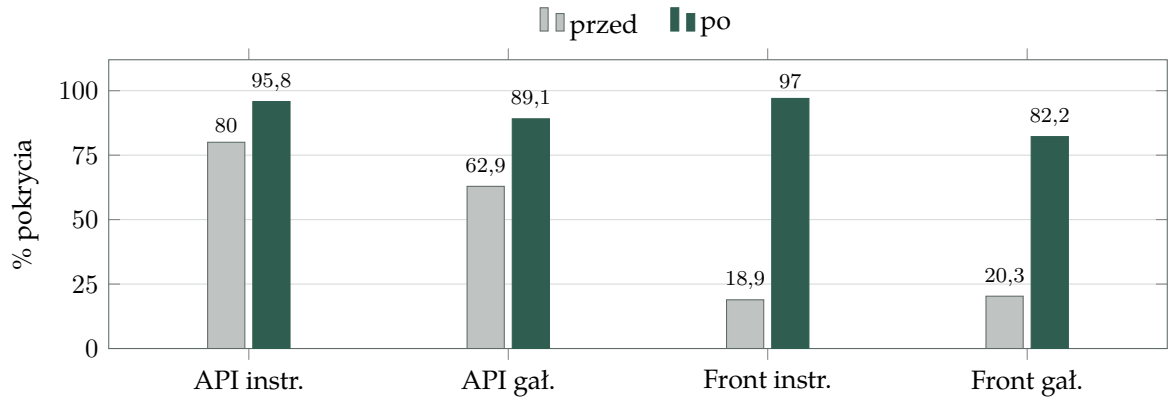
12.2 Pokrycie kodu

Pokrycie mierzono narzędziem V8. Progi są zapisane w konfiguracji Vitest obu pakietów, więc spadek poniżej nich kończy `pnpm test:unit` błędem. Poza pomiarem pozostają tylko pliki startowe procesu: `index.ts` w API i `main.tsx` we froncie.

	API				Front			
	instr.	gał.	funk.	linie	instr.	gał.	funk.	linie
Przed uzupełnieniem testów[b]	80,0	62,9	81,3	84,0	18,9	20,3	18,8	19,1
Po uzupełnieniu	**95,8**	**89,1**	**95,8**	**99,1**	**97,0**	**82,2**	**96,1**	**97,7**
Próg	90	85	90	95	90	80	90	90

[b] Pierwszy pomiar obejmował również pliki startowe, więc wartości są nieco zaniżone.

Tabela 12: Pokrycie kodu w procentach



12.3 Obszary o słabszym pokryciu

- `AuthProvider.tsx`, 62,5% gałęzi: nie jest sprawdzone odmontowanie komponentu przed odpowiedzią `/api/auth/me`.
- `ListDetailPage.tsx`, 75,6% gałęzi: w oknie edycji pozycji nie są zmieniane pola ilości i jednostki, wysyłane są tylko istniejące wartości.
- `apps/api/src/http.ts`, 60% gałęzi: brak przypadku, w którym błąd walidacji nie ma komunikatu.

13 Wyniki testów systemowych

ID	Przypadek	Czas	Wynik
TS-01	Logowanie istniejącym kontem	0,8 s	**zaliczony**
TS-02	Właściciel zarządza listą, osoba ze współdzielenia zmienia tylko status	2,6 s	**zaliczony**
TS-03	Weryfikacja ręczna w przeglądarce	—	**zaliczony** po poprawce D5

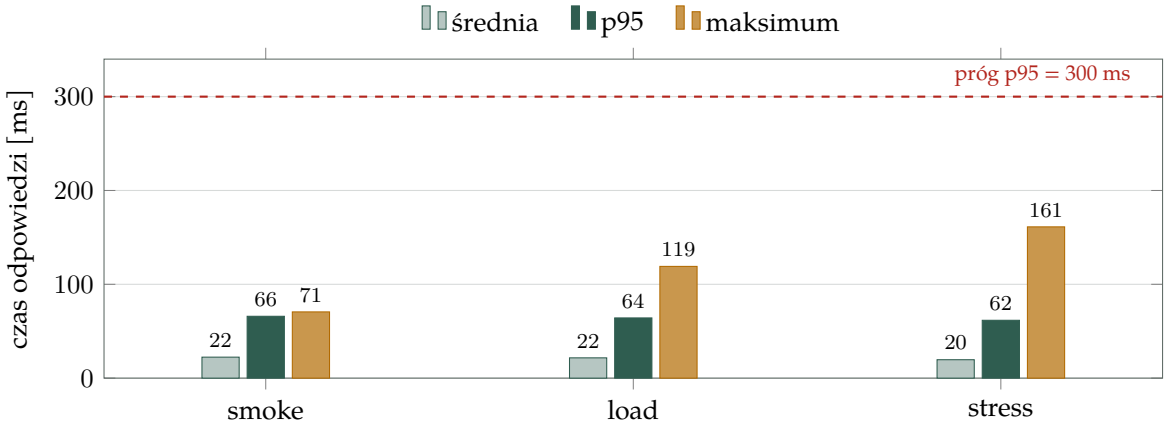
Tabela 13: Wyniki testów systemowych

Weryfikacja ręczna wykryła błąd użyteczności D5: kalendarz wyboru daty zostawał otwarty po wybraniu dnia i zasłaniał przycisk „Utwórz”. Po poprawce kalendarz zamyka się po wyborze, a scenariusz przeszedł w całości, łącznie z utrwaleniem stanu 1/1 po odświeżeniu strony.

14 Wyniki testów wydajnościowych

Miara	TW-01 smoke	TW-02 load	TW-03 stress
Wirtualni użytkownicy (maks.)	1	10	34
Iteracje	38	787	2 985
Żądania HTTP	115	1 465	5 712
Przepustowość [żądania/s]	11,25	72,66	225,85
Średni czas [ms]	22,36	21,61	19,62
Mediana [ms]	2,18	3,84	9,21
p90 [ms]	64,00	61,34	56,18
p95 [ms] (próg 300)	**65,91**	**64,16**	**61,64**
Maksimum [ms]	70,58	119,08	161,22
Błędy HTTP (próg 1%)	0,00%	0,00%	0,00%
Sprawdzenia zaliczone	114/114	1 464/1 464	5 711/5 711
Wynik	**zaliczony**	**zaliczony**	**zaliczony**

Tabela 14: Wyniki testów wydajnościowych k6



14.1 Analiza

- Przy 20-krotnym wzroście przepustowości (z 11 do 226 żądań/s) p95 pozostał prawie stały, około 62–66 ms. Próg 300 ms został spełniony z prawie pięciokrotnym zapasem.
- Rośnie tylko ogon rozkładu: maksimum wzrosło z 71 do 161 ms. To pierwszy sygnał kolejkowania żądań przy większej współbieżności.
- Rozkład jest dwumodalny: mediana wynosi 2–9 ms, a p90 już 56–64 ms. Najpewniej szybkie są odczyty i zmiany statusu, a wolne — logowania liczące bcrypt. Obecne progi nie rozdzielają tych operacji, więc hipotezy nie da się potwierdzić z wyników.
- Test obciążeniowy nie znalazł punktu nasycenia: przy 34 VU nie wystąpił żaden błąd.
- Klient k6 i API działały na tej samej maszynie, przez pętlę zwrotną i w trybie deweloperskim (`tsx`). Wyniki są optymistyczne względem wdrożenia sieciowego.

15 Wyniki audytu bezpieczeństwa

15.1 Testy automatyczne i zależności

Przypadki TB-01–TB-07 zakończyły się wynikiem pozytywnym. `pnpm audit` nie zgłosił żadnej znanej podatności. Wcześniej audyt wykrył podatność krytyczną w pakiecie `tinypool` (zależność Vitest 3) i średnią GHSA-82fw-gwwq-j7x9 w Vitest poniżej 4.1.11. Obie usunięto przez aktualizację Vitest do 4.1.11 i nadpisanie wersji `tinypool` w `pnpm-workspace.yaml` (defekt D4).

15.2 OWASP ZAP baseline

Skan ZAP 2.17.0 objął `http://127.0.0.1:3000`. Wynik: **66 reguł zaliczonych, 1 ostrzeżenie, 0 alertów FAIL**.

Reguła	Alert	Adresy	Waga
10049	Storable and Cacheable Content	`/` i `/robots.txt` (odpowiedzi 404)	**niski**

Tabela 15: Alerty OWASP ZAP

Pająk ZAP znalazł tylko 3 adresy, bo API nie serwuje strony pod `/`, a skan był nieuwierzytelniony. Wynik potwierdza poprawne nagłówki odpowiedzi publicznych, ale nie obejmuje tras za logowaniem ani frontendu.

15.3 Ustalenia przeglądu kodu

ID	Ustalenie	Waga
B1	**Klucz limitu logowań pochodzi z nagłówka** `X-Forwarded-For` (`apps/api/src/middleware/rateLimit.ts`). Nagłówek ustawia klient, więc atakujący omija limit, zmieniając jego wartość przy każdej próbie. Bez nagłówka wszyscy klienci trafiają do wspólnego koszyka `local`, więc jeden atakujący może zablokować logowanie wszystkim użytkownikom. TI-12 potwierdza, że o kluczu decyduje nagłówek.	**średni**
B2	**Tokeny sesji są przechowywane jawnie.** Token jest kluczem tabeli `sessions`. Wyciek pliku bazy pozwala przejąć każdą aktywną sesję na okres do 7 dni.	**średni**
B3	**Brak** `Cache-Control: no-store` **w odpowiedziach JSON** (alert ZAP 10049). Pośrednie bufory mogą przechowywać odpowiedzi API.	**niski**
B4	**Brak rejestrowania zdarzeń bezpieczeństwa** (OWASP A09). Nieudane logowania, odpowiedzi 403 i 429 nie trafiają do dziennika, więc atak nie zostawia śladu.	**niski**
B5	**Limit logowań jest trzymany w pamięci procesu.** Resetuje się przy restarcie i nie działa wspólnie dla kilku instancji.	**niski**
B6	**Ochrona przed CSRF opiera się wyłącznie na** `SameSite=Lax`**.** API odczytuje JSON bez sprawdzania nagłówka `Content-Type`. Atak z tej samej witryny (np. z subdomeny) nie byłby zablokowany.	**informacyjny**
B7	**Ograniczony zasięg DAST**: 3 adresy, bez sesji i bez frontendu.	**informacyjny**

ID	Ustalenie	Waga
B8	**Polityka haseł**: minimum 8 znaków, bcrypt z kosztem 10, bez sprawdzania haseł z wycieków.	informacyjny

15.4 Mechanizmy ocenione pozytywnie

- wszystkie zapytania SQL są parametryzowane; TB-04 potwierdza brak wstrzyknięcia;
- hasła haszowane bcrypt; ten sam komunikat i porównanie z fikcyjnym skrótem dla nieznanego konta ograniczają wyliczanie kont (TI-02);
- token sesji ma 256 bitów losowości, ciasteczko `HttpOnly`, `SameSite=Lax`, a w produkcji `Secure`;
- cudze zasoby zwracają 404, więc nie da się sprawdzić istnienia listy (TB-06);
- przesłane pliki są sprawdzane po typie, rozmiarze i sygnaturze bajtowej (TB-07);
- limit ciała żądania 3 MB i ogólny komunikat przy błędzie 500 (TI-13);
- lockfile przechodzi politykę łańcucha dostaw pnpm, a audyt zależności jest czysty.

16 Defekty wykryte w trakcie testów

ID	Wykrył	Opis	Rozwiązanie
D1	testy integracyjne	błędne ścieżki importu (`../` zamiast `./`) w `context.ts`, `http.ts`, `uploads.ts`; API nie startowało	poprawione ścieżki
D2	kontrola typów	jednostka pozycji miała typ `string` zamiast listy dozwolonych wartości	schemat `z.enum(UNITS)`
D3	testy komponentów	DOM przenikał między testami; brak `ResizeObserver` w jsdom	sprzątanie po teście i atrapy w `setup.ts`
D4	`pnpm audit`	podatność krytyczna w `tinypool` i średnia w Vitest	Vitest 4.1.11, nadpisanie `tinypool`
D5	test ręczny	kalendarz zostawał otwarty i zasłaniał przycisk „Utwórz”	zamknięcie kalendarza po wyborze dnia
D6	przygotowanie testów k6	limit 10 logowań na minutę uniemożliwiał pomiar	zmienna `LOGIN_RATE_LIMIT_MAX` dla środowiska testowego
D7	przygotowanie skanu ZAP	kontener nie widział API na `127.0.0.1`	uruchomienie ZAP w sieci hosta

Tabela 16: Defekty wykryte i usunięte

17 Wnioski i zalecenia dla zespołu developerskiego

17.1 Wnioski

1. Funkcjonalność jest kompletna i stabilna: 78 przypadków testowych przechodzi bez błędów, a reguła współdzielenia jest potwierdzona na poziomie domeny, API i przeglądarki.
2. Piramida testów działa zgodnie z założeniem: większość defektów (D1–D4) wykryły szybkie testy niższych poziomów, a test ręczny wykrył problem użyteczności (D5), którego nie

opisywał żaden test automatyczny.

3. Wydajność z dużym zapasem spełnia wymaganie R8 przy badanym obciążeniu. Granica możliwości systemu nie została jeszcze ustalona.
4. Najistotniejsze ryzyko bezpieczeństwa leży w konfiguracji limitu logowań (B1), a nie w obsłudze danych wejściowych. Ta jest solidna.

17.2 Zalecenia

Nr	Zalecenie	Priorytet
Z1	Kluczować limit logowań adresem gniazda; `X-Forwarded-For` uwzględniać tylko za skonfigurowanym, zaufanym proxy. Dodać drugi limit na konto (e-mail). Dopisać test, w którym zmiana nagłówka nie omija limitu (B1).	wysoki
Z2	Zapisywać w bazie skrót SHA-256 tokenu sesji zamiast samego tokenu; przy weryfikacji porównywać skróty (B2).	średni
Z3	Dodać `Cache-Control: no-store` do odpowiedzi JSON API, zostawiając obecne ustawienie dla zdjęć (B3).	średni
Z4	Rozszerzyć DAST: uwierzytelniony skan ZAP z sesją testową, `zap-api-scan` na podstawie opisu OpenAPI i skan zbudowanego frontendu (B7).	średni
Z5	Uruchamiać testy jednostkowe z progiem pokrycia, integracyjne, bezpieczeństwa i E2E w CI przy każdej zmianie; k6 i ZAP cyklicznie. Zapisywać kolejne cykle TDD jako osobne commity, aby historia pokazywała przejście od czerwonego do zielonego testu.	średni
Z6	Rejestrować nieudane logowania, odpowiedzi 403 i 429 z identyfikatorem konta i adresem (B4).	niski
Z7	W k6 dodać progi per operacja (`http_req_duration{name:login}`), test długotrwały (soak) i stress do pierwszych błędów. Mierzyć zbudowane API na osobnej maszynie.	niski
Z8	W testach systemowych dodać Firefox i WebKit oraz scenariusze negatywne: złe hasło, wejście obcej osoby na adres cudzej listy.	niski
Z9	Podnieść pokrycie gałęzi frontu: odmontowanie `AuthProvider`, zmiana ilości i jednostki w oknie edycji pozycji. Usunąć `users.findById` i `requireToggle` albo użyć ich w kodzie produkcyjnym — dziś wywołują je tylko testy.	niski
Z10	Przy skalowaniu przenieść limit prób do współdzielonego magazynu (np. Redis); rozważyć koszt bcrypt 12 i sprawdzanie haseł z wycieków (B5, B8).	niski

Przed wdrożeniem produkcyjnym należy wykonać co najmniej zalecenia Z1 i Z2. Pozostałe zalecenia poprawiają odporność i zasięg testów, ale nie blokują wydania.

A Odtworzenie wyników

Polecenie	Działanie
`pnpm install`	instalacja zależności
`pnpm test:unit`	testy jednostkowe i integracyjne z progami pokrycia
`pnpm test:integration`	tylko testy integracyjne
`pnpm --filter @shopping/e2e install:browsers`	instalacja Chromium dla Playwright
`pnpm test:e2e`	testy systemowe
`pnpm test:perf`	seed i trzy scenariusze k6 na osobnym API
`pnpm test:security`	testy bezpieczeństwa i `pnpm audit`
`pnpm dev:api`, `pnpm test:zap`	API i skan OWASP ZAP baseline
`tectonic docs/raport/sprawozdanie.tex`	kompilacja tego dokumentu

Tabela 17: Polecenia